# Algoritmo di Dijkstra

| | |
|---|---|
| Materia | Algoritmi |
| Anno | Secondo Anno |
| Data | @January 9, 2025 |
| Studiato | Done |

Ricordiamo lo pseudo-codice:

```
Dijkstra(G, s, w):
    Initialize-Single-Source
    S = empty
    Q = empty
    for each vertex v in V do
        insert(Q, v)
    while Q is not empty do
        u = extract-min(Q)   // u tale che u.d sia minimo
        S = union(S, {u})
        for each v in Adj(u) do
            relax(u, v, w)
            if the call of relax decreased v.d then
                decrease-key(Q, v, v.d)
```

dove $Q$ è una coda di priorità. Si estrae il nodo di minima priorità dalla coda, lo si inserisce nell'insieme $S$ e per ogni nodo nella lista di adiacenza del vertice si rilassano i vertici.
La coda
$Q$ è una coda di min-priorità (rappresentabile con un **min-heap**). In particolare, la coda memorizza considerando come chiave i loro valori di distanza $v.d$ dalla sorgente $s$.

`Initialize-single-source` richiede $O(|V|)$, il primo for $O(|V|)$, il while $O(|V|)$ ma dipende dal tipo di coda, e il secondo for richiede $O(Adj(u))$.

L'algoritmo mantiene l'invariante $Q = V \setminus S$ dalla linea $6$. Possiamo dimostrare la correttezza dell'algoritmo di Dijkstra.

## Correttezza dell'algoritmo di Dijkstra

> *L'algoritmo di Dijkstra eseguito su un grafo orientato $G = (V, E)$ con una funzione di peso $w : E \to \mathbb{R}_0^+$ e vertice sorgente $s$ termina con $v.d = \delta(s, v)$ per ogni $v \in V$.*

Dimostriamo che all'inizio di ogni iterazione del ciclo while di righe $6 - 12$, $v.d = \delta(s, v)$ per ogni vertice $v \in S$. Basta dimostrare questo perché l'algoritmo termina quando $S = V$, quindi avremo che $v.d = \delta(s, v)$ per ogni vertice $v \in V$ dovuto alla procedura di rilassamento. Dimostriamo ciò per induzione sul numero di iterazioni già avvenute nel ciclo while, che è uguale a $|S|$ all'inizio di ogni iterazione.

Per il caso base possiamo considerare in realtà due casi:

- $|S| = 0$, ciò implica che $S$ è vuoto quindi la proprietà è banalmente vera in quanto vale per ogni elemento di $S$;
- $|S| = 1$, allora $S = \{ s \}$ in quanto il primo elemento che viene estratto è quello con priorità minima, ossia la sorgente. In questo caso la proposizione è vera perché $s.d = 0 = \delta(s, s)$.

Consideriamo il passo induttivo. Supponiamo come ipotesi induttiva:

- $v.d = \delta(s, v)$ per ogni $v \in S$;
- $|S| \geq 1$.

Ora viene estratto $u$ da $V \setminus S = Q$ e viene aggiunto ad $S$. Dobbiamo dimostrare che $u.d = \delta(s, u)$. Abbiamo due casi:

- se non ci sono cammini da $s$ ad $u$, allora per la no-path property $u.d = \infty = \delta(s, u)$;
- se esiste un cammino da $s$ ad $u$, allora sia $y$ il primo vertice su un cammino minimo da $s$ ad $u$ che non sia in $S$ e sia $x$ il suo predecessore.

Notiamo che $y$ potrebbe anche coincidere con $u$. Sicuramente $y$ **non appare dopo** $u$ (banalmente, infatti $y$ è il primo vertice in un cammino verso $u$), e **poiché i pesi sono non negativi**, allora $\delta(s, y) \leq \delta(s, u)$. Inoltre, `extract-min` in riga $7$ ha restituito $u$ in quanto $u$ ha la stima minore, ossia $u.d$ è minimo in $Q$, quindi $u.d \leq y.d$. Poiché $x$ è predecessore di $y$ e $y$ è il primo vertice che non è in $S$, allora $x \in S$ e per ipotesi induttiva si ha che $x.d = \delta(s, x)$. Quando $x$ è stato aggiunto a $S$ l'arco $(x, y)$ è stato rilassato quindi per la convergence property vale $y.d = \delta(s, y)$. Infine, per la upper bound property, $u.d \geq \delta(s, u)$.

Ricapitolando si ha:

- $\delta(s, y) \leq \delta(s, u)$;
- $\delta(s, u) \leq u.d$;
- $u.d \leq y.d$;
- $y.d = \delta(s, y)$.

La catena di disuguaglianze:

$$\delta(s, y) \leq \delta(s, u) \leq u.d \leq y.d = \delta(s, y)$$

dimostra che $u.d = \delta(s, u)$ e per la upper bound property questo valore non cambia ulteriormente. Quindi la tesi è dimostrata.

## Corollario

> *Dopo aver eseguito l'algoritmo di Dljkstra su un grafo orientato e pesato $G$ con pesi non negativi e sorgente $s$, il sottografo dei predecessori $G_\pi$ è un albero dei cammini minimi radicato ad $s$.*

Ciò è un'immediata conseguenza della dimostrazione precedente tramite la predecessor subgraph property.

## Analisi della complessità

```
Dijkstra(G, s, w):
    Initialize-Single-Source
```

```
S = empty
Q = empty
for each vertex v in V do
   insert(Q, v)
while Q is not empty do
   extract-min(Q)   // u tale che u.d sia minimo
   S = union(S, {u})
   for each v in Adj(u) do
      relax(u, v, w)
      if the call of relax decreased v.d then
         decrease-key(Q, v, v.d)
```

L'algoritmo mantiene la coda di priorità $Q$ invocando `insert` alla linea $7$ e `decrease-key` alla linea $12$. Invoca `insert` e `extract-min` una volta per vertice.

Per ogni vertice $u$ aggiunto ad $S$, ogni arco in $Adj[u]$ è esaminato nel ciclo for di righe $9-12$ esattamente una volta durante tutto l'algoritmo. Quindi si guardano gli archi di tutto il grafo una sola volta: il ciclo for quindi viene eseguito esattamente $|E|$ volte.

Il tempo di esecuzione di Dijkstra è:

$$O(|V|(\text{tempo-Extract-min}) + |E|)$$

Questo tempo dipende dalla specifica implementazione della coda di priorità. Se usiamo un array lineare e quindi enumeriamo i vertici $1, \ldots, |V|$, così che il vertice $v$ si trovi in $v-$esima posizione, allora il tempo di `extract-min` è $O(|V|)$ e globalmente si ha:

$$O(|V|^2 + |E|) = O(|V|^2)$$

Se implementiamo $Q$ con un min-heap, allora il tempo per `extract-min` è $O(\log |V|)$ e quindi si ha un tempo:

$$O((|V| + |E|) \log |V|) = O(|E| \log |V|)$$

supponendo $|E| = \Omega(|V|)$. Questo perché si ha la chiamata a funzione `decrease-key` dopo il rilassamento dei vertici della lista di adiacenza di $u$, che richiede eventualmente di sistemare l'heap.
Se si usa una matrice di adiacenza, si ottiene
$O(|V|^2 \log |V|)$.

## All-Pairs-Shortest-Path

L'input del problema è un grafo orientato $G = (V, E)$ pesato con funzione peso $w : E \to \mathbb{R}$. L'obiettivo è trovare un cammino minimo da $u$ a $v$ per tutte le coppie $u, v \in V$.

L'output di un algoritmo che risolve questo problema è una matrice $n \times n$ tale che sia le colonne sia le righe sono indicizzate dai vertici di $V$ e se prendo l'elemento di posto $i, j$ della matrice risultante si ottiene la lunghezza del cammino dal vertice $i$ al vertice $j$.

Possiamo risolvere il problema anche applicando l'algoritmo di Bellman-Ford $|V|$ volte, ma possiamo fare di meglio. Inoltre, se $Im(w) = A \subseteq \mathbb{R}_0^+$, possiamo utilizzare Dijkstra e usare un min-heap, ottenendo un tempo $O(|V||E| \log |V|)$, minore di $O(|V|^3)$ se il grafo è sparso.

Se ci sono archi di peso negativo, usando Bellman-Ford si ha $O(|V|^2|E|)$ che diventa $O(|V|^4)$ se il grafo è denso.

Enumeriamo i vertici di $G$ con $V = \{ 1, 2, ..., n \}$. L'input al problema è rappresentato dalla matrice $W = (w_{ij})$, con:

$$w_{ij} = \begin{cases} 0 & i = j \\ w(i,j) & i \neq j \land (i,j) \in E \\ \infty & i \neq j \land (i,j) \notin E \end{cases}$$

Il grafo non può avere cicli di peso negativo, ma può avere archi di peso negativo.

La matrice dei precedessori è $\Pi = (\pi_{ij})$ dove $\pi_{ij}$ è uguale a:

- $\text{nil}$ se $i = j$ oppure se non ci sono cammini da $i$ a $j$;
- predecessore di $j$ in un cammino minimo da $i$ a $j$.

La $i$-esima riga di $\Pi$ induce un albero di cammini minimi radicato a $i$. Possiamo stampare i cammini minimi con la funzione seguente:

```
Print-All-Pairs-Shortest-Path(Pi, i, j):
    if i = j then
        print i
    else if pi(ij) = nil then
        print "no path from i to j"
    else do
        Print-All-Pairs-Shortest-Path(Pi, i, pi(ij))
        print j
```

Osserviamo tuttavia che un sottocammino di un cammino minimo è minimo, quindi i problemi shortest-path hanno la proprietà di sottostruttura ottima. Possiamo quindi utilizzare la programmazione dinamica. Ci serve prima una formulazione ricorsiva del problema.

Indichiamo con il simbolo $l_{ij}^{(r)}$ il peso di un cammino minimo da $i$ a $j$ che contiene al più $r$ archi. Possiamo dire che:

$$l_{ij}^{(0)} = \begin{cases} 0 & i = j \\ \infty & i \neq j \end{cases}$$

Se $r \geq 1$, possiamo considerare un cammino che va da $i$ a $k$ con $r-1$ archi e poi considerare l'arco $(k, j)$. Allora:

$$l_{ij}^{(r)} = \min_{1 \leq k \leq n} \{ l_{ik}^{(r-1)} + w_{kj} \}$$

Un cammino minimo, tuttavia, è un cammino semplice, in quanto non ci sono cicli negativi. Inoltre, un cammino minimo ha al più $n-1$ archi dati $n$ vertici. Di conseguenza, possiamo dire che:

$$\delta(i, j) = l_{ij}^{(n-1)} = l_{ij}^{(n)} = l_{ij}^{(n+1)} = \dots$$